

Deployment And Operations

Date: 2026-03-24

Deployment Scope

This document describes how to run the current coursework prototype. It is suitable for local development and Codespaces-style environments. It shouldn't be used as a deployment guide when used in official production.

Technology Stack

Frontend:

- React 19
- TypeScript
- Vite
- React Router

Backend:

- Django 6
- Django REST Framework
- `django-cors-headers`
- WhiteNoise
- ReportLab

Database:

- PostgreSQL when `DB_HOST` is set
- SQLite fallback when `DB_HOST` is not set

Application Routes In The Current Build

Backend routes:

- `/`: Django home page
- `/accounts/login/`: Django username/password login
- `/admin/`: Django admin
- `/api/`: room, message, activity, summary, and final-answer APIs
- `/api/facilitator/`: facilitator-facing APIs for activities, dashboard, summaries, and agents

Frontend routes:

- `/:` login page
- `/rooms:` learner hub
- `/room/:code:` learner room page
- `/room/:code/activities:` learner activity catalogue
- `/room/:code/activity:` learner activity workspace
- `/room/:code/summary:` learner summary page
- `/facilitator:` facilitator dashboard
- `/facilitator/rooms/:code/summary:` facilitator summary page
- `/facilitator/activities:` facilitator activity management page

Environment Variables

Backend variables used by the project:

```
SECRET_KEY=<django secret>
DEBUG=True
ALLOWED_HOSTS=localhost,127.0.0.1
CSRF_TRUSTED_ORIGINS=http://localhost:5173,http://127.0.0.1:5173
DB_HOST=127.0.0.1
DB_PORT=5432
DB_USER=postgres
DB_NAME=postgres
DB_PASSWORD=12345
DB_SSLMODE=require
```

Frontend variable:

```
VITE_API_BASE_URL=/
```

Notes:

- if `DB_HOST` is not there, the backend uses SQLite at `backend/db.sqlite3`
- the `SECRET_KEY` fallback in `backend/mysite/settings.py` is for development only, should not be used in official production

Preferred Setup: Codespaces

The repository includes a setup script intended for Codespaces:

```
chmod +x setup_env.sh
source ./setup_env.sh
```

The script:

- installs backend dependencies
- installs and starts PostgreSQL
- sets database environment variables for the current shell session
- runs migrations
- loads starter activity fixtures
- creates `frontend/.env`

After the script finishes:

```
cd backend
python manage.py runserver
```

In a second terminal:

```
cd frontend
npm install
npm run dev
```

Manual Local Setup

Backend

```
cd backend
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
python manage.py migrate
python manage.py loaddata message_board/fixtures/activities.json
python manage.py runserver
```

If you want PostgreSQL instead of SQLite, create `backend/.env` with the database values shown above before running the server.

Frontend

```
cd frontend
npm install
```

Create `frontend/.env` containing:

```
VITE_API_BASE_URL=/
```

Then start the frontend:

```
npm run dev
```

How To Run Tests

Backend tests:

```
cd backend  
pip install pytest pytest-django  
pytest
```

Run a single backend test file:

```
cd backend  
pytest message_board/tests/test_views.py
```

Frontend checks:

```
cd frontend  
npm install  
npm run lint  
npm run build
```

Operational Workflow

Typical development startup sequence:

1. Start the backend from `backend/`
2. Start the frontend from `frontend/`
3. Open the frontend URL printed by Vite
4. Sign in as a learner or facilitator from the frontend login page
5. Use Django auth pages separately if you need admin-style username/password access

Authentication Notes

- learners and facilitators are created using the frontend temporary login

- the temporary login API only accepts `learner` and `facilitator`
- Django username/password login is available at `/accounts/login/`
- Django admin is available at `/admin/`

Maintainer Notes

In this handover, "maintainer" means the developer or team taking over the project after submission.

Key files and directories:

Path	Purpose
<code>backend/manage.py</code>	Django management entry point
<code>backend/mysite/settings.py</code>	Django settings, database config, CORS, CSRF, auth redirects
<code>backend/message_board/models.py</code>	Main data model
<code>backend/message_board/views.py</code>	Core room and activity APIs
<code>backend/message_board/agent_rules.py</code>	Automated intervention rules and configurable thresholds
<code>backend/message_board/summary_service.py</code>	Session summary generation
<code>backend/message_board/pdf_generator.py</code>	PDF export logic
<code>backend/message_board/fixtures/activities.json</code>	Seeded activity data
<code>backend/facilitator_page/views.py</code>	Facilitator APIs for activities, agents, and summaries
<code>frontend/src/pages/</code>	Main frontend pages
<code>frontend/src/routes/router.tsx</code>	Frontend route definitions
<code>setup_env.sh</code>	Codespaces-oriented setup script

Activity And Rule Maintenance

Activities can be maintained in two ways:

- through the facilitator activity management UI
- by editing `backend/message_board/fixtures/activities.json` and reloading it with `python manage.py loaddata message_board/fixtures/activities.json`

How To Create A New Activity

There are two ways to create a new activity in the current build.

Option 1: Use The Facilitator UI

This is the safest route for normal maintenance because it validates the main fields before saving.

1. start the backend and frontend
2. sign in as a facilitator

3. `open /facilitator/activities`
4. enter the activity name, description, and activity type
5. add at least one phase
6. for each phase, provide:
 - a phase name
 - a prompt shown to learners during that phase
 - a positive duration
7. adjust agent settings if the activity needs different nudging thresholds
8. save the activity

The current facilitator UI expects:

- `name` to be at least 2 characters
- at least one phase
- every phase to have a non-empty `name`
- every phase to have a non-empty `prompt`
- every phase to have a positive duration

The UI currently works best with the common phase names:

- `understand`
- `propose`
- `critique`
- `decide`

Those names are not strictly required by the backend, but using them reduces the chance of bugs.

Option 2: Add It In Fixture Data

1. `edit backend/message_board/fixtures/activities.json`
2. add a new `message_board.activity` entry with a unique `pk`
3. include `name`, `description`, `activity_type`, and `phases`
4. optionally include `agent_settings`
5. reload the fixture:

```
cd backend
python manage.py loaddata message_board/fixtures/activities.json
```

Required Activity Structure

An activity record is made up of:

- `name`: short readable title
- `description`: optional guidance shown in the catalogue
- `activity_type`: one of `problem-solving`, `discussion`, or `design critique`
- `phases`: JSON array describing the activity flow

- `agent_settings`: optional per-activity rule overrides

Each phase should include:

- `name`
- `prompt`
- `time_limit_minutes`

Common extra fields already used by the project include:

- `duration_seconds`: used by the facilitator UI when editing
- `turn_limit`: included in seeded fixtures
- `assessment_criteria`: supported by the facilitator UI

Example activity payload:

```
{
  "name": "Cloud Migration Debate",
  "description": "Compare migration options for a legacy service.",
  "activity_type": "discussion",
  "phases": [
    {
      "name": "understand",
      "prompt": "Summarise the current system and its constraints.",
      "time_limit_minutes": 5
    },
    {
      "name": "propose",
      "prompt": "Suggest realistic migration approaches.",
      "time_limit_minutes": 8
    },
    {
      "name": "critique",
      "prompt": "Challenge the risks, costs, and trade-offs of each option.",
      "time_limit_minutes": 8
    },
    {
      "name": "decide",
      "prompt": "Agree the preferred migration plan and justify it.",
      "time_limit_minutes": 5
    }
  ],
  "agent_settings": {
    "evidence": {
      "enabled": true,
      "unsupported_claims_before_nudge": 2
    }
  }
}
```

After Creating The Activity

After adding a new activity, verify that:

1. it appears in the learner activity catalogue
2. it can be selected for a room
3. the activity starts correctly
4. the phase prompt changes as the timer moves through phases
5. summaries still generate after the activity finishes

Agent rules live in:

```
backend/message_board/agent_rules.py
```

This file contains:

- default threshold settings in `DEFAULT_AGENT_SETTINGS`
- helper functions such as `get_agent_config()` and `phase_posts()`
- room-state rules in `check_room_state_rules()`
- post-triggered rules in `check_post_rules()`

When changing agent behaviour:

1. edit or add the rule in `agent_rules.py`
2. register it in the relevant dispatcher
3. verify that the affected activity settings still make sense

Per-activity overrides are stored in `Activity.agent_settings`.

If a maintainer adds a brand new rule, this can be done by:

1. add a default config block for the new rule key in `DEFAULT_AGENT_SETTINGS`
2. create a `check_<rule_name>_rule(...)` function in `backend/message_board/agent_rules.py`
3. decide whether it is a room-state rule or a post-triggered rule
4. register it in `check_room_state_rules()` or `check_post_rules()`
5. make sure it creates interventions with a `rule_name` so summaries and reporting can count it correctly
6. add or update tests in `backend/message_board/tests/test_agent_rules.py`
7. if the rule needs per-activity changing, add that key to the relevant `Activity.agent_settings` JSON

Example `Activity.agent_settings` override:

```
{
  "evidence": {
    "enabled": true,
    "unsupported_claims_before_nudge": 2,
    "min_interval_seconds": 60
  },
  "my_new_rule": {
    "enabled": true,
    "threshold": 4,
    "cooldown_seconds": 180
  }
}
```

The `agent_settings` JSON only overrides keys that already exist in the rule defaults. In the program, that means a new rule should normally be added to `DEFAULT_AGENT_SETTINGS` first, then enabled or changed per activity through `Activity.agent_settings`.

Useful Django Commands

Create a superuser:

```
cd backend
python manage.py createsuperuser
```

Apply migrations:

```
cd backend
python manage.py makemigrations
python manage.py migrate
```

Start the agent timer:

```
cd backend
python manage.py agent_timer
```

Reset rooms and non-superuser users:

```
cd backend
python manage.py reset_rooms_users --yes
```

Use the reset command carefully. It deletes data.

Operational Limitations

- the current implementation is geared to coursework demonstration and development, not production hosting
- no production process manager, monitoring, or backup automation is included